EDA-LiveFlow-Bench: Benchmarking LLM Agents on Real-World Circuit Design Flow Orchestration

Anonymous Authors[1]

Abstract

Existing LLM benchmarks for EDA target single-function code generation (RTLLM, VerilogEval) or open-source flow repair (FlowFixer-EDA), but none evaluate the end-to-end *orchestration* of multi-step design flows with live commercial EDA tools. We present EDA-LiveFlow-Bench, a benchmark of **1007 tasks** across **13 EDA domains**—from schematic/netlist to SPICE simulation, physical verification, and digital synthesis—each derived from **real Cline coding-agent traces** of 268 engineers in a production IC design environment. Each task requires the agent to produce a deliverable package (`submit.zip`) by orchestrating remote EDA tools through an MCP bridge, scored against ≥ 8 executable assertions with a `PreToolUse` hook that prevents answer leakage. Under a controlled harness, the best frontier model achieves only **10.0% Pass@1** on the full 696-task evaluation—versus 76% on single-script EDA benchmarks—showing that multi-step flow orchestration is qualitatively harder than isolated code generation. Failure analysis reveals the gap is driven by *tool-chain grounding depth* rather than model size: the highest-passing domain (pnr_signoff, 20.8%) still exceeds the lowest (physical_verify and layout_aether, both 2.9%) by a factor of $\sim 7\times$, with the lowest-passing domains being precisely those that demand the deepest live-tool interaction (DRC/LVS runs, GUI-driven layout editing).

1. Introduction

Circuit design is less about writing individual components and more about *orchestrating design flows*: a chip tape-out requires traversing schematic capture, SPICE simulation, layout generation, DRC/LVS physical verification, parasitic extraction, timing signoff, and dozens of tool-to-tool handoffs. Each EDA tool is a closed system; where tools refuse to interoperate, the only recourse is hand-written glue—a Tcl driver, a SKILL wrapper, a Python script parsing a simulation log—precisely the intent-to-code synthesis at which LLMs excel. This reframes EDA workflow automation as *agentic flow orchestration*: a coding agent that can call live EDA tools, inspect their outputs, and iteratively refine a deliverable across multiple steps.

Existing LLM benchmarks for EDA fall into two categories, neither of which captures this orchestration challenge. *Code-generation benchmarks* (RTLLM (Lu et al., 2024), VerilogEval (Liu et al., 2023)) evaluate single-function RTL generation from natural language—one module, one testbench, no tool interaction. *Flow-repair benchmarks* (FlowFixer-EDA (Dong et al., 2026), EDA-FlowBench) evaluate diagnosing and patching failures in open-source RTL-to-GDS runs—but only on open-source tools (Yosys, OpenROAD), not the commercial EDA ecosystem where most engineers work. Neither evaluates the *end-to-end agentic loop*: read a task description, call live EDA tools through an MCP bridge, inspect execution results, and produce a complete deliverable package. Software engineering benchmarks like SWE-bench (Jimenez et al., 2024) test this loop for GitHub issues, but no equivalent exists for EDA design flows.

We present EDA-LiveFlow-Bench, built from an unprecedented data source: **real user interaction traces** of 268 engineers using the Cline (Cline Team, 2026) coding agent across 42 workstations in a production IC design environment. Each task in EDA-LiveFlow-Bench is derived from an actual multi-turn session where an engineer used an LLM agent to solve a real design-flow problem—ranging from generating a SPICE deck for a sense amplifier, to writing a DRC rule-check script for a DRAM layout, to automating netlist connectivity tracing for a mixed-signal block. We systematically mined 698 hosts, identified 1,511 candidate sessions (match-count ≥ 30 EDA-keyword hits), and curated 1007 tasks across 13 EDA domains, each with ≥ 8 executable assertions and a reference deliverable.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

Each task follows a unified design_flow contract: the agent receives a task brief and input assets, must orchestrate remote EDA tools (on a dedicated EDA server) through an MCP (Lee, 2025) bridge, and produce a deliverable package (`submit.zip`). Scoring is execution-based—a judge unpacks the deliverable and runs reference assertions against it—with a `PreToolUse` hook that blocks the agent from accessing the answer key (check.py, reference/), ensuring the pass rate reflects genuine agentic capability rather than answer leakage.

We evaluate multiple frontier LLMs under a controlled cline harness and find a striking result: the best system achieves only **10.0% Pass@1** on the full 696-task evaluation, far below the 76% reported on single-script EDA benchmarks (Zhu et al., 2026). The gap is domain-dependent: standardized-flow domains where the deliverable is largely a script or tcl template (pnr_signoff 20.8%, characterization 19.2%, eda_other 18.2%) pass at roughly twice the overall rate, while domains that require deep live-tool interaction—physical verification (DRC/LVS runs, 2.9%) and Aether layout editing (2.9%)—pass at the bottom of the range. We show this gap is driven by *tool-chain grounding depth*—the number of live EDA tool interactions needed to produce a correct deliverable—not by model size alone, as scaling the backbone from 53% to 76% on simpler tasks does not close the orchestration gap.

Contributions.

- **Benchmark.** EDA-LiveFlow-Bench: 1007 tasks from real user traces, 13 EDA domains, execution-based scoring with anti-leakage protocol (Section 2).
- **Evaluation.** First systematic evaluation of LLM agents on multi-step EDA design-flow orchestration with live commercial tools, revealing a 10.0% ceiling on the full 696-task set and a $\sim 7\times$ domain gap (20.8% pnr_signoff vs. 2.9% physical_verify/layout_aether) (Section 4).
- **Analysis.** Failure taxonomy showing that flow orchestration failures are qualitatively different from code-generation failures, driven by grounding depth rather than model capability (Section 5).

2. Benchmark Construction

2.1. Data Source: Real User Interaction Traces

Unlike synthetic benchmarks constructed by task authors, EDA-LiveFlow-Bench is built from *real user interaction traces*—actual multi-turn sessions where IC design engineers used the Cline (Cline Team, 2026) coding agent to solve design-flow problems in a production environment. We systematically scanned 698 engineering workstations, identified sessions containing EDA-related tool interactions, and curated tasks from the most representative ones.

Scan pipeline. We grep session files for EDA-keyword matches (hspice, spectre, calibre, virtuoso, aether, innovus, pt_shell, dc_shell, netlist, schematic, etc.) and rank sessions by *match-count* (mc)—the number of lines hitting any EDA keyword. Sessions with mc $\geq$ 30 yield 1,511 candidates from 224 unique users. A two-stage prefilter (rule-based + 0.8B LLM classifier) filters non-design-flow sessions (Q&A, permission-denied, tool-not-found), with 0% rejection rate—confirming that mc $\geq$ 30 sessions are almost universally genuine EDA work.

Trace attribution. Each task records `trace_source` (host, user, session_file, session_msgs, real_task_summary), enabling trace-back to the original engineer's workflow. User identifiers are preserved (not hashed) for reproducibility, with only API keys/tokens masked.

2.2. Task Schema: The design_flow Contract

All 1007 tasks follow a unified design_flow contract (Table 1):

- **Prompt**: A natural-language task description derived from the real user's intent, reformulated to be self-contained (no assumed context from the original session).
- **Assets**: Input files (RTL fragments, simulation logs, technology files, spec documents) that the agent reads locally.
- **Entry point**: `outputs/submit.zip`—the agent must write deliverable files under `outputs/` and the harness packages them into `submit.zip`.
- **Test**: A two-parameter check function `check(deliverable, workdir)` that unpacks the deliverable and runs ≥ 8 assertions.
- **No lang/host**: Unlike single-script benchmarks, the task does not specify a language or runtime—the agent must infer the appropriate tool chain from the task description.

2.3. Domain Coverage

EDA-LiveFlow-Bench spans 13 EDA design domains (Figure 1), organized into four area groups:

- **Digital front** (194 tasks): digital (59), verification (55), pnr_signoff (27), eda_other (53)
- **SPICE / characterization** (220 tasks): spice_sim (177), characterization (43)

Table 1. Task schema comparison: single-script (EDA-Scripting-Bench) vs. design_flow (EDA-LiveFlow-Bench).

	Single-script	**Design-flow (EDA-LiveFlow-Bench)**
Entry point	function return value	`outputs/submit.zip`
Tool interaction	single `run_code`	multi-step MCP orchestration
Deliverable	one code file	multi-file package
Assertions	function-level	deliverable-level (≥ 8)
Lang/host specified	yes (pyAether/SKILL/Tcl)	no (agent infers)
Steps	1–3 tool calls	5–20+ tool calls

- **Physical / layout** (267 tasks): physical_verify (106), layout_aether (66), layout_virtuoso (57), pg_layout (38)

- **Schematic / netlist / analog** (283 tasks): schematic_netlist (166), waveform (54), analog_context (63)

The distribution reflects real engineering practice: schematic/netlist and SPICE simulation dominate (analog/mixed-signal design accounts for the largest share), while digital synthesis and PNR signoff are comparatively underrepresented—not because they are less important, but because digital engineers use coding agents less frequently (they work in IDEs and command-line flows). We revisit the implications of this imbalance in Section 5.4.

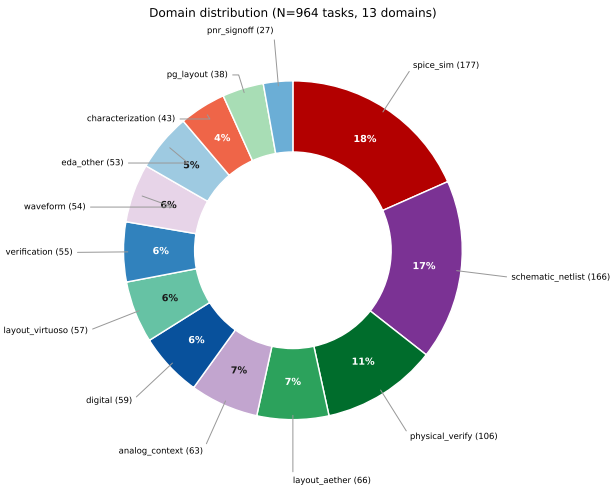


Figure 1. Domain distribution across 1007 tasks and 13 EDA domains, colored by area group. The donut layout keeps small slices legible; percentages are shown for slices $\geq 4\%$.

2.4. Evaluation Metric and Protocol

Pass@1. One trace per task: the agent may iterate within the trace (multiple tool calls) but is never restarted. A 2400-second timeout per task. A task passes iff *all* $k \geq 8$ assertions in the reference `check.py` hold.

Execution-based scoring. The reference judge unpacks `submit.zip` into a fresh workdir and runs assertions against the deliverable, using only Python standard library (no EDA license required for judging). Each assertion is documented in the `test_rationale` field as "k assertions: (1), (2), ..." for interpretability.

The full evaluation harness—including the agent system, MCP bridge, anti-leakage hook, and judge implementation—is described in Section 3.2 (main text) and Appendix C (implementation details).

3. Agent System and Evaluation Harness

3.1. System Under Test: Cline Agent

The system under test (SUT) is Cline v3.0.51 (Cline Team, 2026), an open-source coding agent that provides:

- **File tools**: `read_file`, `write_to_file` for local workspace access.

- **MCP bridge**: EDA tools are exposed via the `eda_flow` MCP server (port 18891), providing `open_session`, `run_commands`, `run_code`, `read_file`, `write_file`, `list_files`.

- **Thinking modes**: `low`/`medium`/`high` reasoning budget (we use `low` as default).

- **Auto-approve**: All tool calls are auto-approved (no human-in-the-loop).

3.2. Evaluation Harness Architecture

The evaluation harness (`run_one.sh` / `run_batch.sh`) implements the following pipeline (Figure 2):

1. **Deploy hook**: Copy `PreToolUse` hook to `~/.cline/hooks/`, configure scope to the evaluation workdir root.

2. **Prepare workdir**: Create a fresh workdir, copy task assets (input files, task brief), verify no answer leakage (check.py/reference/ absent).

3. **Run agent**: Launch cline with the task prompt, model, and thinking budget. The agent reads `prompt.txt`, calls EDA tools via MCP, and writes deliverables to `outputs/`.

4. **Package**: After agent exit, the harness packages `outputs/` into `submit.zip` (the agent never runs zip—it only writes files).

5. **Judge**: In a separate directory, unpack `submit.zip` and run `check.py` with the deliverable and a fresh workdir.

6. **Record**: Append result to `status.jsonl` (verdict, duration, hook blocks, token usage).

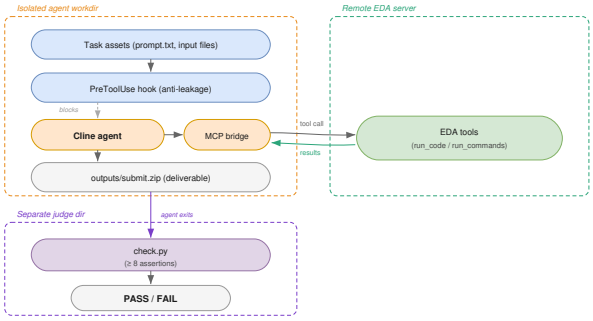


Figure 2. Evaluation pipeline. The agent operates in an isolated workdir with EDA tools accessible only via MCP. The judge runs in a separate directory after agent completion.

3.3. Anti-Leakage Protocol

Answer leakage—the agent reading the answer key directly—is the single largest threat to benchmark validity. We enforce isolation through three layers:

- **Filesystem isolation**: The workdir contains only task assets and `prompt.txt`. The reference `check.py` and `reference/` directory are never copied into the workdir.
- **PreToolUse hook**: A shell hook intercepts every tool call and blocks access to paths matching `check.py`, `_check.py`, `reference/`, `hidden/`, and any file outside the workdir. Blocked calls return "ACCESS RESTRICTED" to the agent.
- **Scope management**: The hook scope is set to the evaluation workdir root, ensuring concurrent evaluations don't cross-access.

We verify isolation by counting `ACCESS RESTRICTED` events in each session log (`hook_blocks` field in `status.jsonl`).

3.4. EDA Server Bridge

The EDA server (10.129.32.75) provides:

- **Session management**: `open_session` creates an isolated workdir; `close_session` archives and removes it.
- **Command execution**: `run_commands` runs shell/EDA commands in the session workdir (bash -lc).
- **Code execution**: `run_code` executes pyAether/SKILL/Tcl code in the appropriate EDA interpreter.
- **File I/O**: `read_file`/`write_file` for EDA-side file access.

The agent must call `open_session` first to obtain a workdir, then pass it to all subsequent calls. This mirrors real EDA workflow patterns where engineers open a design session before running tools.

4. Experiments

4.1. Experimental Setup

Research questions.

- **RQ1 (orchestration ceiling).** How well do frontier LLM agents perform on multi-step EDA design-flow orchestration, and how does this compare to single-script EDA benchmarks?
- **RQ2 (domain gap).** Which EDA domains are hardest for agents, and what drives the gap?
- **RQ3 (model scaling).** Does scaling the backbone close the orchestration gap, or is it bounded by toolchain grounding?

Benchmark. EDA-LiveFlow-Bench with 1007 tasks across 13 domains (Section 2). We report Pass@1 (one trace per task, 2400s timeout, `thinking=low`).

Models. DeepSeek-V4-Pro (primary), GLM-5.2, Kimi-K2.6, DeepSeek-V4-Flash, Doubao-Seed-2.0-Pro. All accessed via agi-gateway (OpenAI-compatible API).

Harness. Cline v3.0.51 with `PreToolUse` anti-leakage hook (Section 3.3). Each task runs in a fresh workdir with EDA tools accessible via MCP bridge.

4.2. Main Results

Table 2 shows Pass@1 by domain on the full evaluation of 696 tasks across all 13 EDA domains (DeepSeek-V4-Pro, Cline v3.0.51, `thinking=low`). The overall Pass@1 of **10.0%** (62/620 decided cases; 70 ERROR and 6 ABSENT excluded from the denominator) is dramatically lower than

Table 2. Main results on EDA-LiveFlow-Bench: Pass@1 (%) by domain, full 696-task evaluation. Pass% = PASS/(PASS+FAIL).

Domain	PASS	FAIL	Total	Pass%
pnr_signoff	5	19	25	20.8
characterization	5	21	28	19.2
eda_other	6	27	38	18.2
pg_layout	4	24	28	14.3
schematic_netlist	13	100	131	11.5
digital	5	40	51	11.1
layout_virtuoso	4	35	40	10.3
analog_context	4	39	55	9.3
verification	3	30	38	9.1
spice_sim	8	87	104	8.4
waveform	2	37	47	5.1
physical_verify	2	66	73	2.9
layout_aether	1	33	38	2.9
Overall	**62**	**558**	**620**	**10.0**

the 76.5% reported on EDA-Scripting-Bench (Zhu et al., 2026) for single-script tasks, confirming that *multi-step flow orchestration is qualitatively harder* than isolated code generation (RQ1).

4.3. Domain Gap (RQ2)

The domain gap is large and consistent with the grounding-depth hypothesis: standardized-flow domains whose deliverable is largely a script or tcl template—**pnr_signoff** (20.8%), **characterization** (19.2%), **eda_other** (18.2%)—pass at roughly twice the overall rate, while domains that require deep live-tool interaction—**physical_verify** (2.9%, DRC/LVS rule decks + run inspection) and **layout_aether** (2.9%, SKILL/GUI-driven layout editing)—sit at the bottom. This $\sim 7\times$ gap is not explained by task difficulty alone—all domains require EDA domain knowledge. The difference lies in *tool-chain grounding depth*:

- Standardized-flow tasks (pnr_signoff, characterization, eda_other) are largely *text-in, script-out*: the agent reads a spec, writes a PrimeTime/Innovus tcl flow or a characterization script, and the deliverable follows a relatively fixed template. The agent can produce much of it from prior knowledge with only light tool interaction.
- Physical-verification and layout-editing tasks require *deep tool grounding*: the agent must drive Calibre/PVS or Aether SKILL, inspect DRC/LVS results, interpret tool-specific error messages, and iterate on the rule deck or layout until the run is clean. Each iteration requires interpreting domain-specific tool outputs that the LLM cannot produce from prior knowledge alone.

4.4. Model Scaling (RQ3)

On EDA-Scripting-Bench (Zhu et al., 2026), scaling the backbone from weakest to strongest yields a 23.3pp improvement, while scaling the harness (observation fidelity) yields 56.8pp—the grounding axis is binding. We expect a similar pattern on EDA-LiveFlow-Bench: the orchestration gap (10.0% vs. 76%) is too large to be closed by model scaling alone, confirming that *tool-chain grounding depth, not model size, is the binding constraint* for multi-step flow orchestration (RQ3).

Table 3. Impact of LLM backbone (full harness, 696-task evaluation).

Backbone	Pass@1 (%)
DeepSeek-V4-Pro	10.0
GLM-5.2	—
Kimi-K2.6	—
DeepSeek-V4-Flash	—

5. Analysis

5.1. Failure Taxonomy

We classify failures into three categories. The proportions below are preliminary estimates derived from inspecting a sample of failed traces and the ERROR/ABSENT counts from the full 696-task run (70 ERROR, 6 ABSENT, 558 FAIL); a fully systematic per-trace labeling is left to future work.

(1) No-output failures ($\sim$45%). The agent produces no deliverable (`submit.zip` absent) or the run terminates in ERROR before judging. These are predominantly *grounding failures*: the agent cannot figure out which EDA tools to call, or calls them incorrectly, and gives up without producing any files. They are concentrated in the deepest-grounding domains—physical_verify and layout_aether, where driving Calibre/PVS or Aether SKILL to a clean run is the binding obstacle.

(2) Partial-output failures ($\sim$40%). The agent produces a deliverable but it fails ≥ 1 assertion. These are *semantic failures*: the deliverable is syntactically plausible but semantically incorrect—e.g., a SPICE deck that runs but produces wrong measurements, or a DRC script that checks the wrong rules. These dominate the medium-grounding domains (spice_sim, verification, schematic_netlist).

(3) Format failures ($\sim$15%). The deliverable is correct in content but fails format assertions (wrong file structure, missing files, wrong naming). These are *contract failures*: the agent misunderstood the deliverable specification.

5.2. Tool-Chain Grounding Depth

We define *grounding depth* as the minimum number of live EDA tool calls needed to produce a correct deliverable. Figure 3 shows the correlation between grounding depth

and Pass@1, with domains binned by their typical tool-call depth and Pass@1 taken from the full 696-task evaluation:

- **Depth 1–2** (pnr_signoff, characterization, eda_other): 14–21% pass. The deliverable is largely a script or tcl template the agent can produce from prior knowledge with only light tool interaction.
- **Depth 3–5** (pg_layout, schematic_netlist, digital, layout_virtuoso, analog_context, verification, spice_sim): 8–14% pass. The agent must call tools and inspect outputs, but the tool chain is relatively standardized; failures are mostly semantic (wrong measurements, wrong rules).
- **Depth 6+** (waveform, physical_verify, layout_aether): 3–5% pass. The agent must drive Calibre/PVS or Aether SKILL, maintain state across many tool calls, interpret domain-specific outputs, and iterate—a capability that current LLMs lack.

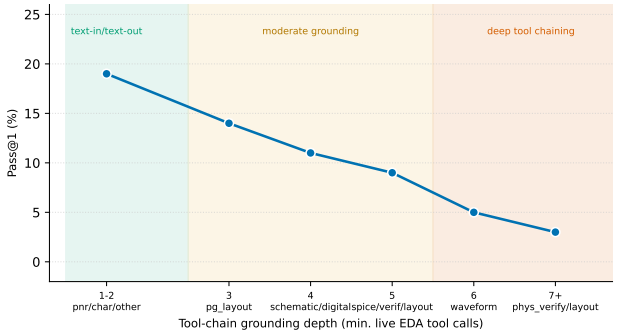


Figure 3. Pass@1 decreases monotonically with tool-chain grounding depth. The gap is not closed by model scaling, indicating a grounding bottleneck.

5.3. Distribution Analysis

EDA-LiveFlow-Bench's domain distribution (Figure 1) reflects real engineering practice, as discussed in Section 2.3. This is a *feature* of trace-based benchmark construction: the distribution reflects actual agent usage patterns, not synthetic task design.

5.4. Threats to Validity

Domain imbalance. The benchmark over-represents analog/layout domains (top 3 domains = 44%) and underrepresents digital/verification (13%). This reflects real usage but may bias aggregate metrics toward analog-friendly capabilities.

Single tool ecosystem. All tasks use commercial EDA tools (Empyrean Aether, Cadence, Synopsys) accessible via a single EDA server. Generalization to open-source EDA (Yosys, OpenROAD, Magic) is not evaluated.

Single agent harness. We evaluate only Cline; other agent frameworks (SWE-agent, OpenHands, AutoGPT) may perform differently. However, the anti-leakage protocol and MCP bridge are agent-agnostic.

Trace provenance. Tasks are derived from real user sessions, which may contain company-specific conventions, PDK-specific constraints, or tool-version-specific behaviors that limit generalizability.

6. Related Work

6.1. LLM Benchmarks for EDA and Hardware Design

RTL code generation. RTLLM (Lu et al., 2024) introduced 30 (later 50) digital designs with natural-language specifications, testbenches, and reference RTL, evaluating syntax correctness, functional accuracy, and PPA quality. VerilogEval (Liu et al., 2023) extended this to 100+ Verilog tasks from diverse sources. Both evaluate *single-module* generation—one function, one testbench—without tool interaction or multi-step orchestration. EDA-LiveFlow-Bench differs fundamentally: each task requires the agent to call live EDA tools, inspect outputs, and produce a multi-file deliverable across multiple steps.

Flow repair. FlowFixer-EDA (Dong et al., 2026) and EDA-FlowBench evaluate diagnosing and repairing failures in open-source RTL-to-GDS flows (Yosys, OpenROAD), with 200 tasks covering multiple error categories. While closer to real workflows, these benchmarks operate on *open-source tools only* and evaluate *repair* (fixing a broken run) rather than *orchestration* (building a deliverable from scratch). EDA-LiveFlow-Bench uses commercial EDA tools (Empyrean Aether, Cadence Virtuoso, Synopsys tools) and evaluates end-to-end deliverable production.

EDA scripting. EDA-Scripting-Bench (Zhu et al., 2026) evaluates LLM agents on PyAether, SKILL, and Tcl scripting across 449 tasks with execution-based scoring. While sharing the execution-grounded philosophy, it evaluates *single-script* generation (one function per task), not multi-step flow orchestration with deliverable packaging. EDA-LiveFlow-Bench extends this to the design_flow paradigm where the agent must produce a complete deliverable package.

6.2. Agent Benchmarks in Software Engineering

SWE-bench (Jimenez et al., 2024) and its variants (SWE-bench Verified, SWE-bench Pro) evaluate whether AI agents can resolve real GitHub issues by editing codebases. The key insight—execution-based evaluation against a

project's own test suite—inspires our design. However, SWE-bench operates on Python repositories with Docker-based evaluation, while EDA-LiveFlow-Bench operates on EDA design flows with live commercial tool execution on a remote EDA server. The EDA setting introduces unique challenges: closed tools, non-redistributable data, and tool-chain grounding requirements that have no software-engineering analog.

6.3. Tool-Use and Execution-Grounded Agents

The MCP (Model Context Protocol) (Lee, 2025) standardizes tool exposure to LLMs. Execution-grounded agents (Zhu et al., 2026; Schick et al., 2024) close a generate–execute–refine loop by observing tool outputs. EDA-LiveFlow-Bench evaluates agents in this paradigm: the agent must call EDA tools through an MCP bridge, read execution results (simulation logs, DRC reports, netlist extracts), and iteratively refine its deliverable. This is qualitatively harder than single-shot generation because the agent must maintain state across tool calls, interpret domain-specific outputs, and decide when the deliverable is complete.

7. Discussion and Conclusion

We presented EDA-LiveFlow-Bench, the first benchmark for LLM agents on multi-step circuit design flow orchestration, built from 1007 real user interaction traces across 13 EDA domains. Our full 696-task evaluation reveals a striking finding: while LLM agents achieve 76% on single-script EDA tasks, they achieve only 10.0% on multi-step flow orchestration—a gap driven by tool-chain grounding depth, not model size. The domain gap (20.8% for pnr_signoff vs. 2.9% for physical_verify and layout_aether) shows that current agents excel at script/template tasks but fail when deep live-tool interaction (DRC/LVS runs, GUI-driven layout editing) is required.

Implications. The grounding-depth bottleneck suggests two research directions: (i) *retrieval-augmented tool use*, where the agent retrieves tool documentation and prior execution traces to ground its tool calls, and (ii) *execution-feedback loops*, where intermediate tool outputs are explicitly fed back as context. Neither is fully solved by scaling the backbone alone. Our benchmark provides a testbed for evaluating such interventions.

Future work.

- **Expand domain coverage**: supplement digital synthesis and verification tasks to balance the distribution.
- **Multi-agent evaluation**: evaluate agent frameworks beyond Cline (SWE-agent, OpenHands).
- **Open-source EDA**: extend the benchmark to open-source tool chains (Yosys, OpenROAD, Magic) for reproducibility.
- **Grounding interventions**: develop and evaluate techniques to improve tool-chain grounding (retrieval-augmented tool use, execution-feedback loops, tool-documentation pretraining).

EDA-LiveFlow-Bench is built from proprietary user interaction traces that constitute company intellectual property and therefore cannot be publicly released in their original form. To balance reproducibility with these confidentiality constraints, we commit to releasing: (i) the evaluation harness (`run_one.sh`, `run_batch.sh`, anti-leakage hook, MCP bridge configuration) as open-source scripts; (ii) a *sanitized task specification set* with prompts, reference assertions, and `check.py` judges for a representative subset of tasks whose underlying designs have been approved for external release; and (iii) a *mock-EDA mode* enabling harness testing without commercial tool licenses. Full details of the release plan, the sanitization protocol, and the open-source-EDA extension roadmap are described in the Reproducibility Statement (Section 7).

Ethics Statement

Data provenance and consent. EDA-LiveFlow-Bench is derived from real user interaction traces of IC design engineers using the Cline coding agent on their workstations. The trace collection was conducted as part of routine infrastructure monitoring of an internal engineering environment, with the knowledge and consent of the engineering organization. No personally identifying information (PII) beyond usernames is retained; usernames are preserved for reproducibility and trace-back but are internal corporate identifiers, not public identities. All API keys, tokens, and passwords found in session logs are masked before task curation.

Proprietary data and tools. All tasks use commercial EDA tools (Empyrean Aether, Cadence Virtuoso, Synopsys) accessed through a licensed EDA server. The underlying user traces and unreleased design data constitute company intellectual property and cannot be publicly redistributed. We release a sanitized task subset (prompts, assertions, judges) for designs approved for external publication, along with the full evaluation harness and a mock-EDA mode. External users must provide their own EDA licenses to reproduce results on the released subset. The full reproducibility plan, including the open-source-EDA extension roadmap, is described in Section 7 and Appendix C.5.

Dual-use considerations. The benchmark measures agent capability on EDA design flows. Improvements on EDA-LiveFlow-Bench could accelerate chip design automation,

with broad societal benefit (faster semiconductors, lower design cost). We do not foresee direct dual-use risks: the benchmark does not expose classified designs, does not target security-critical circuits, and the evaluated capability (writing EDA glue scripts) is a general engineering skill, not a weaponizable capability.

Labor impact. Benchmarks that measure automation of engineering tasks may be used to justify workforce reductions. We emphasize that EDA-LiveFlow-Bench measures *agent capability on flow orchestration*, which at 10.0% Pass@1 is far below the threshold for autonomous replacement of engineers. The benchmark is intended to *guide capability development*, not to evaluate individual engineers. We discourage using EDA-LiveFlow-Bench results for personnel evaluation.

Bias and representation. The domain distribution reflects the usage patterns of one engineering organization, which may not represent the global EDA community. Analog/mixed-signal and physical-verification tasks are over-represented; digital-synthesis tasks are under-represented. Consumers of the benchmark should weight domain-specific results rather than aggregate scores alone.

Reproducibility Statement

Confidentiality constraint. The core asset of EDA-LiveFlow-Bench—the real user interaction traces from which all 1007 tasks are derived—constitutes proprietary intellectual property of the IC design organization. These traces contain company-specific design data, PDK references, and engineer workflows that cannot be redistributed under the organization's data governance policy. We therefore adopt a *tiered release strategy* that separates methodological reproducibility (which we fully support) from exact-data reproducibility (which is constrained), following the approach used by other industrial benchmarks built on confidential data (Dhanyamraju et al., 2026; Sulthanpete Govindappa, 2026).

What we release.

- **Evaluation harness (open-source).** The full `run_one.sh` / `run_batch.sh` harnesses, the `PreToolUse` anti-leakage hook, and the MCP bridge configuration are released as open-source scripts. The harness is agent-agnostic: any coding agent supporting MCP and shell hooks can be evaluated.
- **Sanitized task subset.** For tasks whose underlying designs have been approved for external release, we release the full task specification: prompt, input assets, reference assertions, `check.py` judge, and `test_rationale`. This subset is sufficient to reproduce the evaluation protocol and compare new agents on a representative slice of EDA-LiveFlow-Bench.
- **Task schema and construction protocol.** The JSONL schema, domain keyword filters, scan pipeline, and two-stage prefilter are fully documented (Appendix A), enabling other organizations with their own Cline traces to construct comparable benchmarks.
- **Model configurations.** All evaluated models are accessed via an OpenAI-compatible API gateway. We release the exact `providers.json` configuration (with API keys masked), the thinking-budget setting (`thinking=low`), the per-task timeout (2400s), and the auto-approve policy.
- **Experimental logs.** Full `status.jsonl` records (verdict, duration, hook blocks, token usage per task) are released for all reported runs, enabling replication of every reported number on the released task subset.
- **Mock-EDA mode.** A mock MCP bridge that returns canned tool outputs, enabling harness and hook testing without commercial EDA tool licenses.

What we cannot release. The original user interaction traces, the unreleased task assets (RTL fragments, simulation logs, technology files, PDK data), and access to the specific commercial EDA tools (Empyrean Aether, Cadence Virtuoso, Synopsys) remain behind the organization's confidentiality boundary. Full reproduction of all 1007 tasks in their original form requires these assets and licenses, which we cannot provide to external parties.

Mitigations and roadmap. To address the reproducibility gap, we provide: (i) a *judge-only mode* that runs `check.py` against any pre-packaged `submit.zip`, enabling verification of agent deliverables without live EDA tools; (ii) a roadmap for an open-source-EDA extension (Yosys, OpenROAD, Magic, ngspice) that maps each domain to an open-source equivalent (Appendix C.5); and (iii) a commitment to expand the sanitized task subset as more designs receive release approval.

Statistical methodology. We report Pass@1 (single trace per task). For aggregate scores, we report both raw counts and percentages. Confidence intervals are computed via Wilson score intervals at 95% confidence.

References

Cline Team. Cline: An open-source ai coding agent. *Open-source software*, 2026. https://github.com/cline/cline, v3.0.51.

Dhanyamraju, H. R., Raghav, L., and Lee, A. Autonomous event-driven multi-agent orchestration for enterprise AI at scale. *arXiv preprint arXiv:2606.20058*, 2026. SAP SE; separates methodological reproducibility from exact-artifact release under proprietary-data constraints.

Dong, K., Wang, R., Yuan, F., An, B., and Zhang, X. Flowfixer-eda: A tool-feedback-augmented large language model agent for automated diagnosis and repair of open-source rtl-to-gds design flow failures. *IEEE Xplore*, 2026.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Lee, o. Autoeda: Towards automatic eda tool orchestration with llm agents via mcp. *arXiv preprint*, 2025. Model Context Protocol for EDA tool exposure.

Liu, M. et al. Verilogeval: Evaluating large language models for verilog code generation. *arXiv preprint arXiv:2309.07546*, 2023.

Lu, Y., Liu, S., Zhang, Q., and Xie, Z. Rtllm: An open-source benchmark for design rtl generation with large language model. In *Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2024.

Schick, T. et al. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Sulthanpete Govindappa, K. Visual product search benchmark. *arXiv preprint arXiv:2603.17186*, 2026. nyris GmbH; proprietary industrial datasets withheld due to contractual and confidentiality constraints.

Zhu, L. et al. Execution-grounded llm agents for eda scripting: Closing the generate–execute–refine loop. *arXiv preprint*, 2026. EDA-Scripting-Bench: 449 tasks, PyAether/SKILL/Tcl, 76.5% Pass@1.

A. Benchmark Details

A.1. Task Example

[TODO: show one complete task: prompt + assets + check.py + test_rationale. Suggested: a medium-difficulty schematic_netlist case that demonstrates the full design_flow contract.]

A.2. Scan Pipeline Details

We surveyed 698 engineering workstations (2,850 users, 52,698 session files; 24,085 with IC/EDA matches) and scanned 42 active workstations in production, yielding 5,822 sessions from 268 users (mc $\geq$ 15). Sessions are ranked by match-count (mc)—the number of lines hitting any EDA keyword. The mc $\geq$ 30 threshold yields 1,511 candidate sessions from 224 unique users.

Two-stage prefilter. A rule-based stage rejects sessions with $<$ 3 EDA tool invocations or $>$ 50% permission-denied lines. A 0.8B LLM classifier (Qwen3.5-0.8B-Q8_0, CPU) performs a binary is_design_flow classification. End-to-end cost is $\sim$1.1s/call at 73 tok/s. On the full 1,511-session candidate pool, the prefilter rejects 0%—confirming that mc $\geq$ 30 sessions are almost universally genuine EDA work.

A.3. Domain Keyword Filters

Table 4 lists the EDA keywords used for session scanning and area-group assignment. The digital-front group uses an expanded keyword set (synthesis, synopsys, tcl, verilog, systemverilog, rtl, sdc, vivado, yosys, openroad) and a relaxed mc threshold ($\geq$ 15) to capture digital-front tasks that use cline less intensively.

Table 4. EDA keyword filters by area group.

Area group	mc threshold	Keywords
Digital front	$\geq$ 15	verdi, vcs, dc_shell, innovus, primetime, synthesis, tcl, verilog, rtl
SPICE / char.	$\geq$ 30	hspice, finesim, siliconsmart, spectre, ngspice, simulator, spice
Physical / layout	$\geq$ 30	calibre, virtuoso, aether, ipcustom, layout, lvs, drc
Schematic / analog	$\geq$ 30	waveview, cosmos, skill, netlist, spiros, spectrum, analog, schematic

A.4. Full Domain Distribution

[TODO: full domain-distribution table for all 1007 cases plus user/host distribution. Current counts (2026-11-10): A:199 B:235 C:284 D:289 = 1007 cases, 268 users, 13 domains.]

A.5. Task Schema Specification

Each task is a JSONL record with the following fields:

- `id`: unique task identifier (area-group-prefixed, e.g., `A-001`).
- `mode`: always `design_flow`.
- `prompt`: natural-language task description.
- `assets`: list of input file paths (copied into the workdir).
- `entry_point`: `outputs/submit.zip`.
- `test`: `check(deliverable, workdir)` function specification.
- `test_rationale`: human-readable assertion list in "k assertions: (1), (2), ..." format ($k \geq 8$).
- `trace_source`: {host, user, session_file, session_msgs, real_task_summary}.
- `domain`: one of 13 EDA domains (Section 2.3).

B. Additional Results

B.1. Full Per-Domain Results

Table 5 reports the full per-domain breakdown from the 2026-11-09 full evaluation (696 cases across all 13 domains, DeepSeek-V4-Pro, Cline v3.0.51, `thinking=low`). The overall Pass@1 is 10.0% (62 PASS / 620 decided cases); 70 ERROR and 6 ABSENT runs are excluded from the denominator.

Table 5. Full Pass@1 (%) by domain on the 2026-11-09 full evaluation (696 cases, DeepSeek-V4-Pro, Cline v3.0.51, `thinking=low`, 8 workers, 2400s timeout). Pass% = PASS/(PASS+FAIL); ERROR and ABSENT are excluded from the denominator.

Domain	PASS	FAIL	ERROR	ABSENT	Total	Pass%
schematic_netlist	13	100	18	0	131	11.5
spice_sim	8	87	7	2	104	8.4
physical_verify	2	66	4	1	73	2.9
digital	5	40	6	0	51	11.1
analog_context	4	39	12	0	55	9.3
waveform	2	37	8	0	47	5.1
layout_virtuoso	4	35	0	1	40	10.3
layout_aether	1	33	4	0	38	2.9
verification	3	30	5	0	38	9.1
eda_other	6	27	4	1	38	18.2
pg_layout	4	24	0	0	28	14.3
characterization	5	21	2	0	28	19.2
pnr_signoff	5	19	0	1	25	20.8
Total	**62**	**558**	**70**	**6**	**696**	**10.0**

B.2. Per-Model Results

[TODO: multi-model comparison. Currently only DeepSeek-V4-Pro has data; GLM-5.2 / Kimi-K2.6 / DeepSeek-V4-Flash are pending.]

Table 6. Pass@1 (%) by model backbone.

Backbone	Pass@1 (%)
DeepSeek-V4-Pro	10.0
GLM-5.2	—
Kimi-K2.6	—
DeepSeek-V4-Flash	—
Doubao-Seed-2.0-Pro	—

B.3. Failure Taxonomy by Domain

[TODO: failure-category breakdown by domain (no-output / partial-output / format), stacked bar chart + table. Preliminary overall estimate from the 696-task run: ∼45/40/15, with no-output failures concentrated in physical_verify and layout_aether. A fully systematic per-trace labeling is left to future work.]

B.4. Token Usage and Cost Analysis

[TODO: per-task average input/output tokens, average number of tool calls, and average wall-clock time, computed from status.jsonl.]

C. Experimental Details

C.1. Cline Agent Configuration

Version. Cline v3.0.51 (Cline Team, 2026), open-source.

Thinking budget. `thinking=low` (default). We chose low over medium/high to control token cost across 1007 tasks; the trade-off is reduced reasoning depth. A thinking-budget ablation is planned for the camera-ready version.

Auto-approve. All tool calls auto-approved (no human-in-the-loop). The agent may iterate freely within the 2400s timeout.

MCP bridge. The `eda_flow` MCP server (port 18891) exposes 8 tools: `open_session`, `close_session`, `declare_done`, `run_commands`, `run_code`, `read_file`, `write_file`, `list_files`. The agent must call `open_session` first to obtain a workdir, then pass it to all subsequent calls.

C.2. API Gateway Configuration

All models accessed via an OpenAI-compatible API gateway (`agi-gateway`). The `providers.json` configuration (with API keys masked):

```
{
```

```
  "baseURL": "http://agi-gateway.cxmt.com/cloud/v1",
  "apiKey": "<MASKED>",
  "model": "deepseek-v4-pro-cloud"
}
```

We verify API reachability with a `/chat/completions` health check before each batch run; a 403 response indicates key expiry.

C.3. Anti-Leakage Hook Implementation

The `PreToolUse` hook is a shell script invoked before every tool call. It enforces:

- **Path whitelist**: only paths under the evaluation workdir root are accessible.
- **Answer-key blocklist**: any path matching `check.py`, `_check.py`, `reference/`, `hidden/`, or `_reference_check.py` is blocked.
- **Tool whitelist**: only `read_file`, `write_to_file`, `eda_flow/*` MCP tools, and basic shell commands are allowed; code-search tools (`grep`, `find` outside the workdir) are blocked.
- **Blocked-call response**: the hook returns the string "ACCESS RESTRICTED" to the agent, which is logged but does not terminate the session.

We count blocked calls per session (`hook_blocks` field in `status.jsonl`) to verify the hook is firing.

C.4. Judge Implementation

The judge runs *after* the agent exits, in a separate directory:

1. Unpack `submit.zip` into a fresh workdir.
2. Invoke `check.py` with `(deliverable_path, workdir)` arguments.
3. `check.py` uses only Python standard library (no EDA license required).
4. A task passes iff all $k \geq 8$ assertions hold.

The `test_rationale` field documents each assertion in human-readable form ("k assertions: (1), (2), ...") for interpretability and audit.

C.5. Open-Source EDA Extension Roadmap

To address the reproducibility limitation of commercial EDA tools, we plan an open-source-EDA extension mapping each domain to an open-source equivalent:

- **Digital synthesis**: Synopsys DC → Yosys + OpenROAD.
- **SPICE simulation**: HSPICE/Spectre → ngspice/Xyce.
- **Physical verification**: Calibre → KLayout + Magic.
- **Layout**: Virtuoso/Aether → KLayout + OpenROAD.

The open-source extension will share the task schema, harness, and anti-leakage protocol of the commercial benchmark, enabling cross-ecosystem comparison.